

Data Carving with XXD, Binwalk and Scalpel

OJE DOMINION MAYOWA

FWSD2511507

Computer and Digital Forensics

MR. Aminu Idris

5 September 2026

LAB OVERVIEW

This laboratory exercise focused on digital forensic data carving, file recovery, and low-level analysis of digital evidence. The lab involved examining hexadecimal data with `xxd` to identify file signatures, using Sleuth Kit (TSK) tools such as `mmls`, `fsstat`, `fls`, `icat`, and `blkcat` to analyze disk images and extract files, and applying Binwalk and Scalpel for embedded-data and deleted-file recovery. Recovered files were then verified using MD5 and SHA-256 hashes to confirm their integrity and authenticity. Overall, the exercise provided practical experience in hexadecimal analysis, filesystem examination, file carving, evidence extraction, recovery, and forensic integrity verification.

OBJECTIVES

- Explain why data carving is required when file-system metadata is missing or unreliable.
- Use `xxd` to examine binary file contents and identify file headers and footers.
- Recognise JPEG file signatures, including the Start of Image (SOI) and End of Image (EOI) markers.
- Reconstruct binary content from a hexadecimal dump.
- Use The Sleuth Kit to examine inode, block, and sector-level data.
- Identify and extract embedded content from files using Binwalk.
- Recover deleted files from a forensic USB image using Scalpel.
- Verify recovered evidence using cryptographic hashing techniques.
- Prepare a professional forensic report documenting commands, screenshots, findings, and limitations.

TOOLS

- Authorized training Evidence
- Kali Linux
- Microsoft Word
- Github, Git
- `Md5sum` and `sha256sum`
- `Xxd`, `Scalpel`, `hashdeep`, `TSK`

METHODOLOGY

The first step is to create the lab environment and download the Authorized training evidence/materials to be used in the lab

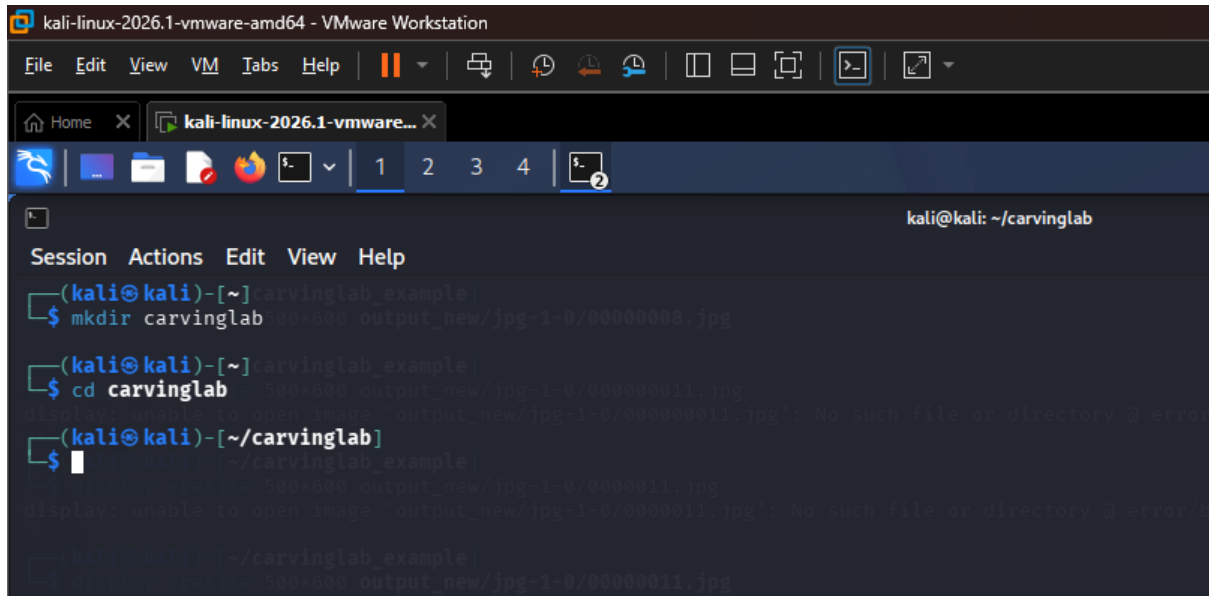


Fig 1.1 Creating Lab Folder

1. Ch01InChap01.dd - Forensic DD Image

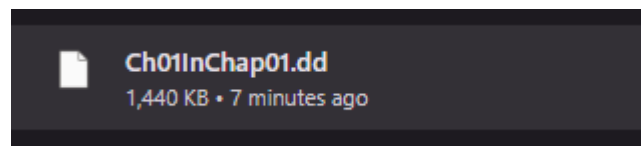


Fig 1.2 Downloaded forensic Image

2. J_ub_law.jpg - JPEG Sample for Hex and Signature Analysis

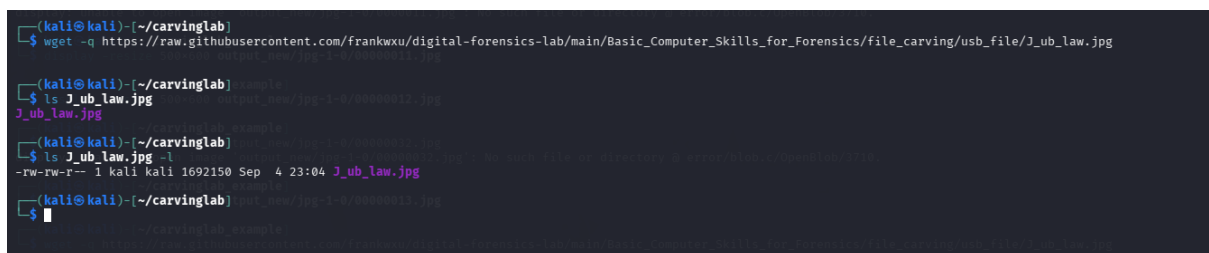


Fig 1.3 Downloaded JPEG

3. 120M.7z - USB Image for Scalpel File Carving

Then we begin the Analysis of lab

Part A - File Signatures and Hex Analysis

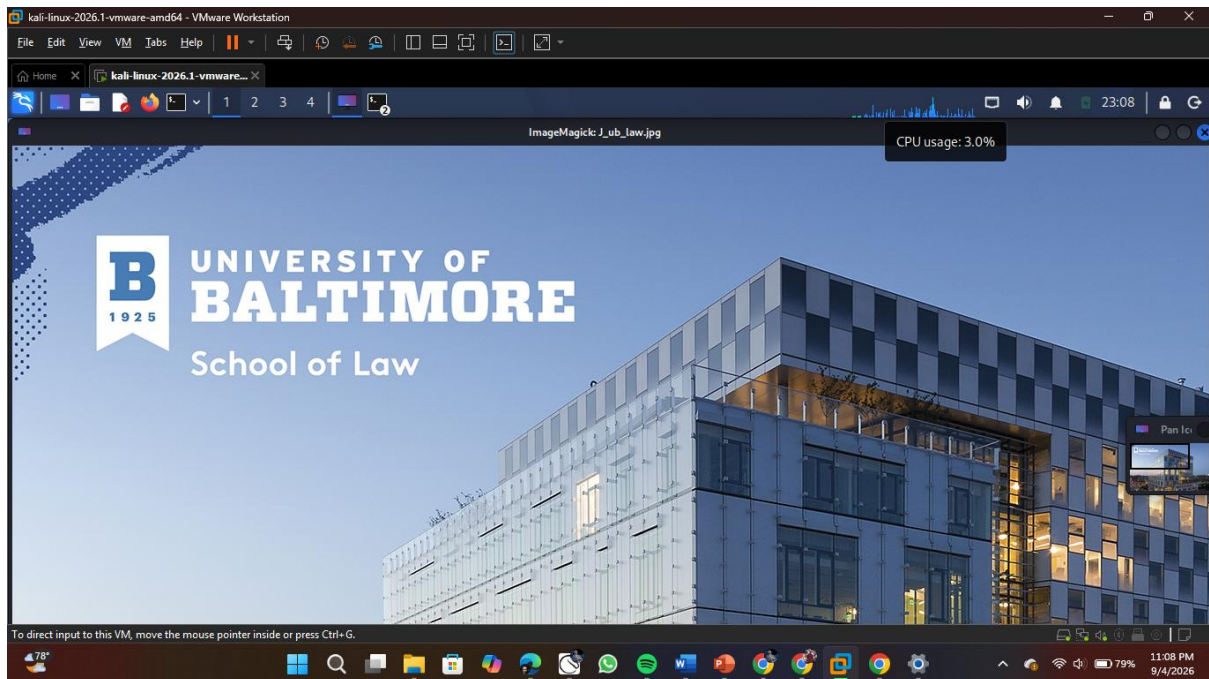


Fig 1.4 Verifying the JPEG Image using the display command

Exploring the image with xxd

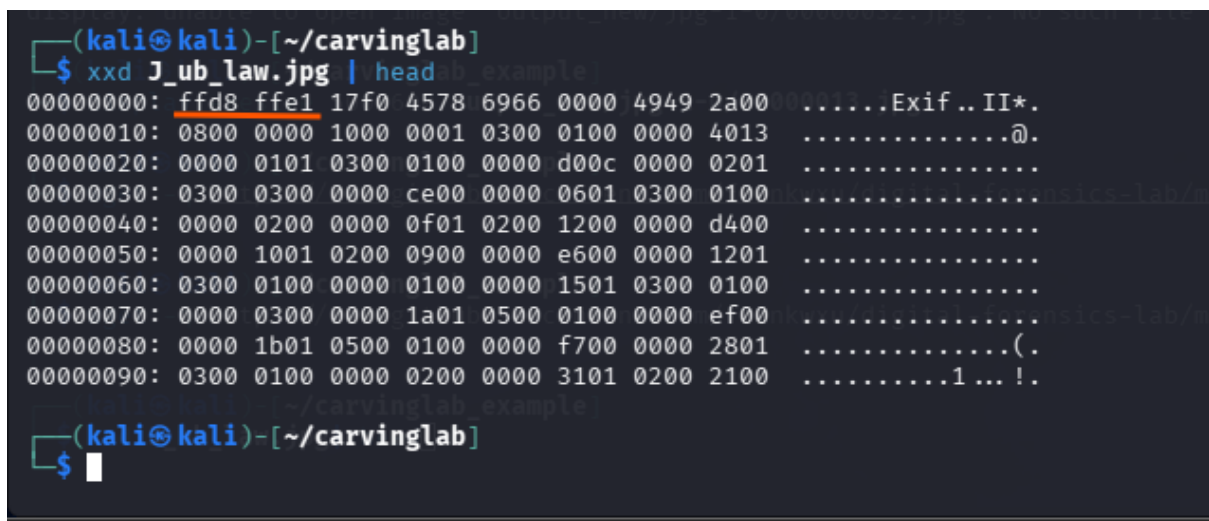


Fig 1.5 Showing us the head of the image

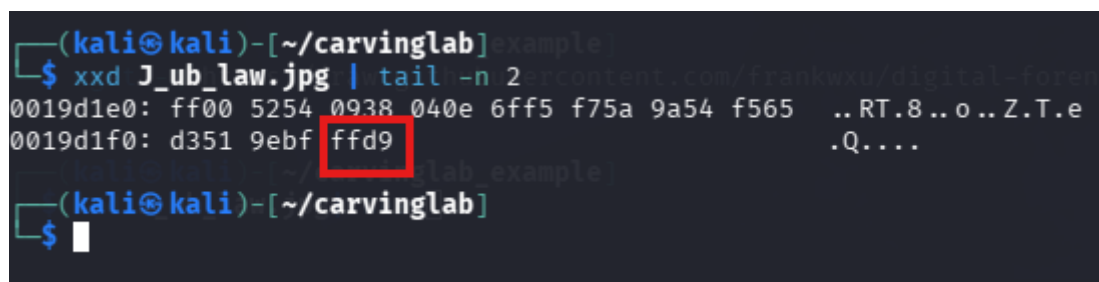


Fig 1.6 Showing us the tail of the image

```

(kali@kali)-[~/carvinglab]
$ xxd -p J_ub_law.jpg | head -5
ffd8ffe117f045786966000049492a000800000010000001030001000000
401300000101030001000000d00c00000201030003000000ce0000000601
03000100000002000000f01020012000000d40000001001020009000000
e60000001201030001000000010000001501030001000000030000001a01
050001000000ef0000001b01050001000000f70000002801030001000000

(kali@kali)-[~/carvinglab]
$

```

Fig 1.7 Converting to a plain Hex dump

We converted to a plain hex dump using the -p option.

Reconstructing the image, we then use the -r option which allows for reverse operations but initially I wasn't getting the expected result because no output file was created and instead I tried to use the image itself which didn't give the expected result.

```

(kali@kali)-[~/carvinglab]
$ xxd -r -p J_ub_law.jpg J_ub_law_reverse_dum.jpg
No such file or directory @ error/dlib.c/openlib:

(kali@kali)-[~/carvinglab]
$ ls J_ub_law_reverse_dum.jpg -l
new/jpg-1-0/00000013.jpg
-rw-rw-r-- 1 kali kali 594 Sep  5 00:12 J_ub_law_reverse_dum.jpg

(kali@kali)-[~/carvinglab]
$ display -resize 300x400 J_ub_law_reverse_dum.jpg
display: Not a JPEG file: starts with 0x80 0x20 `J_ub_law_reverse_dum.jpg' @ error/jpeg.c/JPEGErrorHandler/378.

```

Fig 1.8 Incorrect Output due to Uncreated .txt file

After putting the output of the plain hex file into an **hexdump.txt** file then the desired result was gotten

```

(kali@kali)-[~/carvinglab]
$ ls
J_ub_law.jpg  J_ub_law_reverse_dum.jpg

(kali@kali)-[~/carvinglab]
$ xxd -p J_ub_law.jpg > J_ub_law_hexdump.txt

(kali@kali)-[~/carvinglab]
$ xxd -r -p J_ub_law_hexdump.txt J_ub_law_reverse_dum.jpg

(kali@kali)-[~/carvinglab]
$ display -resize 300x400 J_ub_law_reverse_dum.jpg

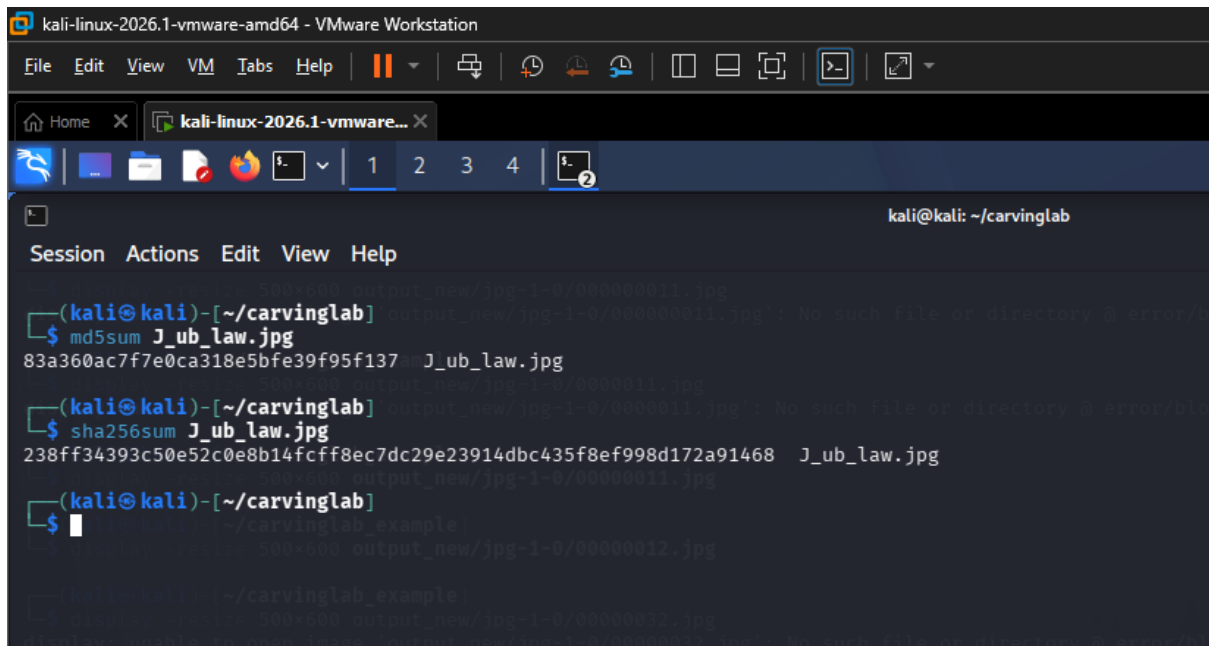
```



Fig 1.9 Correct result of J_ub_law_reverse_dum.jpg

Comparing the hash Values for the original picture and the reversed plain hexdump image (reconstructed file)

Original image



```
kali@kali: ~/carvinglab
$ md5sum J_ub_law.jpg
83a360ac7f7e0ca318e5bfe39f95f137 J_ub_law.jpg
$ sha256sum J_ub_law.jpg
238ff34393c50e52c0e8b14fcff8ec7dc29e23914dbc435f8ef998d172a91468 J_ub_law.jpg
```

Fig 1.10 Hash values for the Original Image

Reconstructed Image

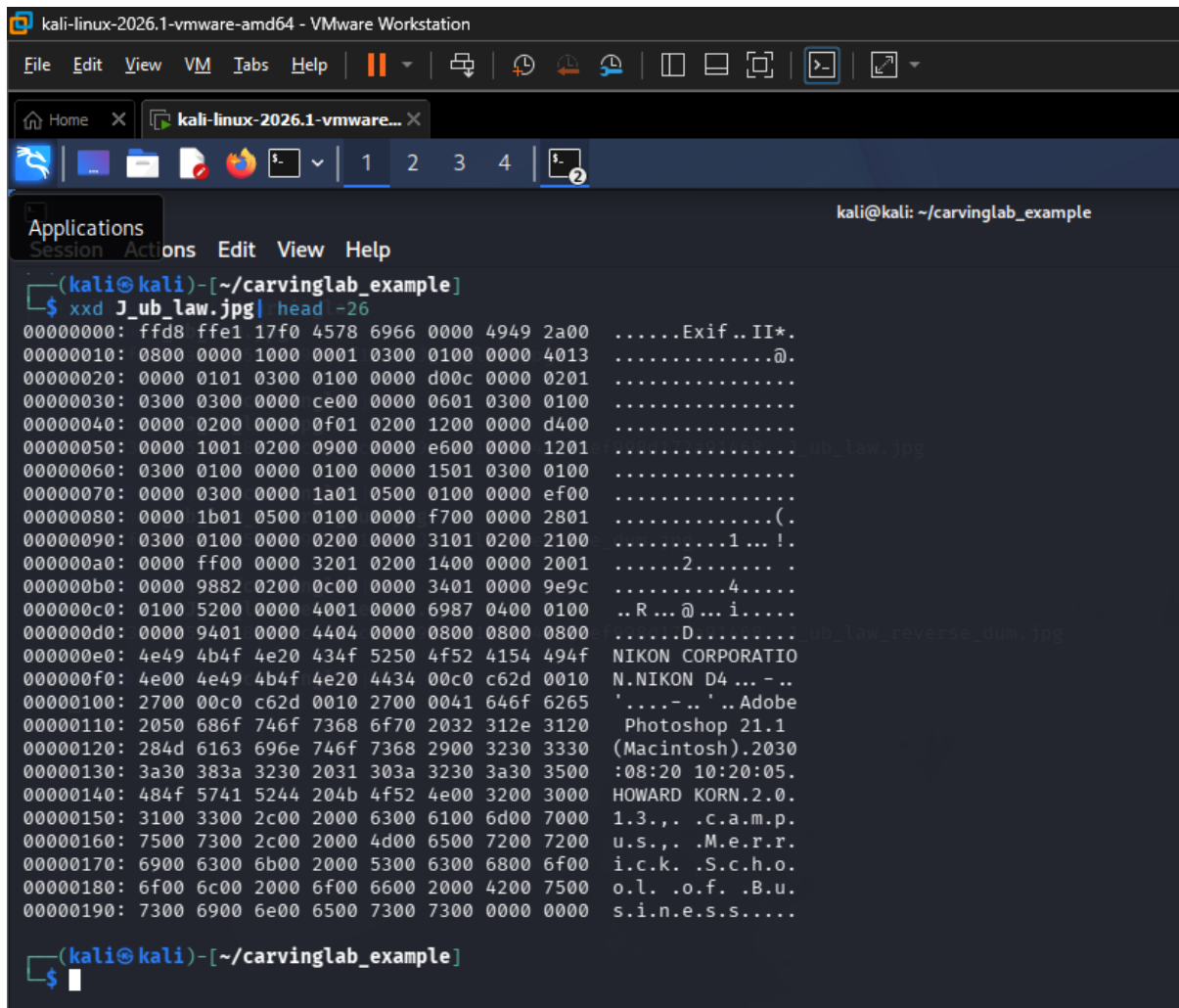


```
kali@kali: ~/carvinglab
$ md5sum J_ub_law_reverse_dum.jpg
83a360ac7f7e0ca318e5bfe39f95f137 J_ub_law_reverse_dum.jpg
$ sha256sum J_ub_law_reverse_dum.jpg
238ff34393c50e52c0e8b14fcff8ec7dc29e23914dbc435f8ef998d172a91468 J_ub_law_reverse_dum.jpg
```

1.11 Hash values for the Reconstructed Image

From this comparison we can see that both images have the same values for their md5sum and sha256sum hash values which shows that there wasn't any alteration to the image and it's content.

Part B - Examine Embedded Metadata-Like Content



```
kali-linux-2026.1-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
kali-linux-2026.1-vmware...
Applications
Session Actions Edit View Help
kali@kali: ~/carvinglab_example
(kali@kali)-[~/carvinglab_example]
$ xxd J_ub_law.jpg | head -26
00000000: ffd8 ffe1 17f0 4578 6966 0000 4949 2a00 .....Exif.. II*.
00000010: 0800 0000 1000 0001 0300 0100 0000 4013 .....@.
00000020: 0000 0101 0300 0100 0000 d00c 0000 0201 .....
00000030: 0300 0300 0000 ce00 0000 0601 0300 0100 .....
00000040: 0000 0200 0000 0f01 0200 1200 0000 d400 .....
00000050: 0000 1001 0200 0900 0000 e600 0000 1201 .....ub_law.jpg
00000060: 0300 0100 0000 0100 0000 1501 0300 0100 .....
00000070: 0000 0300 0000 1a01 0500 0100 0000 ef00 .....
00000080: 0000 1b01 0500 0100 0000 f700 0000 2801 .....(
00000090: 0300 0100 0000 0200 0000 3101 0200 2100 .....1...!
000000a0: 0000 ff00 0000 3201 0200 1400 0000 2001 .....2.....
000000b0: 0000 9882 0200 0c00 0000 3401 0000 9e9c .....4.....
000000c0: 0100 5200 0000 4001 0000 6987 0400 0100 ..R...@...i....
000000d0: 0000 9401 0000 4404 0000 0800 0800 0800 .....D.....ublaw_reverse_dump.jpg
000000e0: 4e49 4b4f 4e20 434f 5250 4f52 4154 494f NIKON CORPORATIO
000000f0: 4e00 4e49 4b4f 4e20 4434 00c0 c62d 0010 N.NIKON D4 ... - ..
00000100: 2700 00c0 c62d 0010 2700 0041 646f 6265 '....-.. ' ..Adobe
00000110: 2050 686f 746f 7368 6f70 2032 312e 3120 Photoshop 21.1
00000120: 284d 6163 696e 746f 7368 2900 3230 3330 (Macintosh).2030
00000130: 3a30 383a 3230 2031 303a 3230 3a30 3500 :08:20 10:20:05.
00000140: 484f 5741 5244 204b 4f52 4e00 3200 3000 HOWARD KORN.2.0.
00000150: 3100 3300 2c00 2000 6300 6100 6d00 7000 1.3.,. .c.a.m.p.
00000160: 7500 7300 2c00 2000 4d00 6500 7200 7200 u.s.,. .M.e.r.r.
00000170: 6900 6300 6b00 2000 5300 6300 6800 6f00 i.c.k. .S.c.h.o.
00000180: 6f00 6c00 2000 6f00 6600 2000 4200 7500 o.l. .o.f. .B.u.
00000190: 7300 6900 6e00 6500 7300 7300 0000 0000 s.i.n.e.s.s....
(kali@kali)-[~/carvinglab_example]
$
```

Fig 1.12 Metadata and Content of the Image

From the metadata like content in the image we have information on the visible camera, software, name, date or other metadata-related strings.

visible camera: Nikon d4

software: Adobe Photoshop 21.1

name: HOWARD KORN

date: 2030

Part C - Direct Data-Unit Examination with The Sleuth Kit

In this section we use some sleuth kit commands to do a data examination with our disk image file Ch0In1chap01.dd

Firstly we begin with the **img_stat** command which is used to show basic information about the forensic image, such as image type and size.


```
(kali㉿kali)-[~/carvinglab]
$ ls -lh
total 8.0M
-rw----- 1 kali kali 1.5M Sep  4 22:52 Ch01InChap01.dd
-rw-rw-r-- 1 kali kali 3.3M Sep  5 00:19 J_ub_law_hexdump.txt
-rw-rw-r-- 1 kali kali 1.7M Sep  4 23:04 J_ub_law.jpg
-rw-rw-r-- 1 kali kali 1.7M Sep  5 00:20 J_ub_law_reverse_dum.jpg

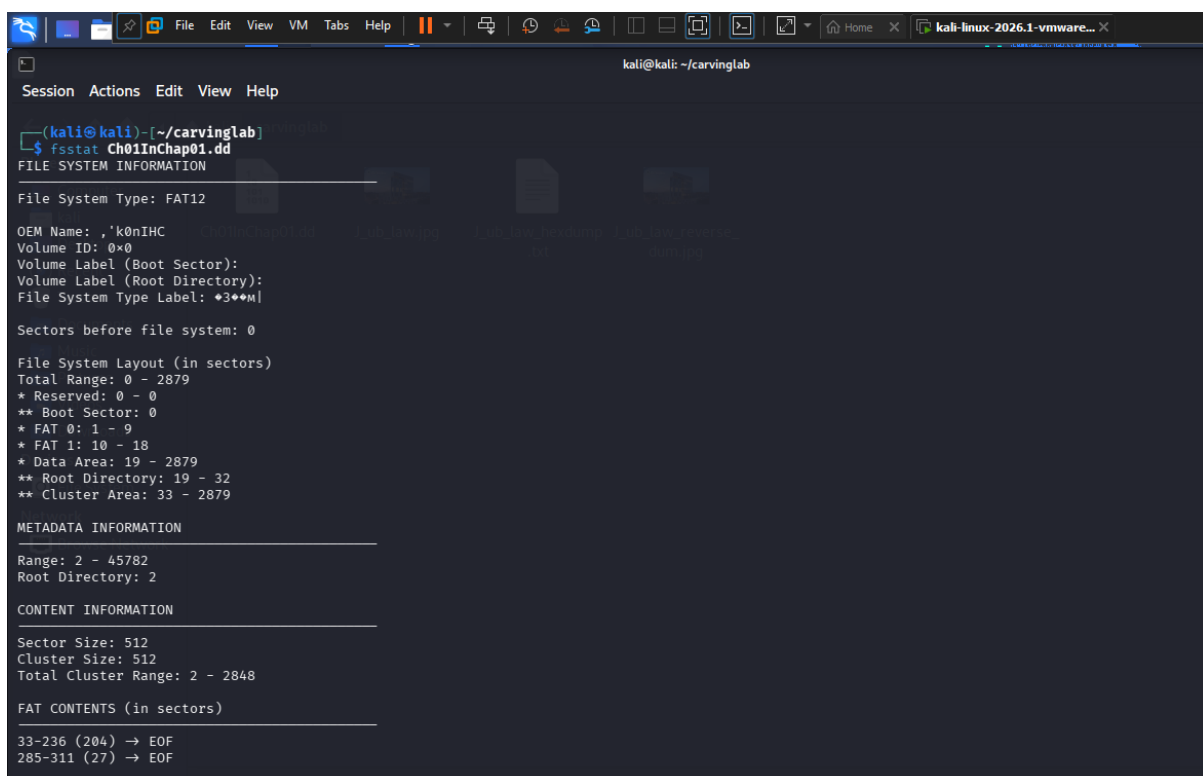
(kali㉿kali)-[~/carvinglab]
$ img_stat Ch01InChap01.dd
IMAGE FILE INFORMATION
-----
Image Type: raw

Size in bytes: 1474560
Sector size:   512

(kali㉿kali)-[~/carvinglab]
$
```

Fig 1.13 Using Img_stat to get information onto the disk

Next we explore the **ffstat** which shows detailed information about the **file system** inside a partition.



```
kali@kali: ~/carvinglab
Session Actions Edit View Help

(kali㉿kali)-[~/carvinglab]
$ ffstat Ch01InChap01.dd
FILE SYSTEM INFORMATION
-----
File System Type: FAT12

OEM Name: ,k0nIHC
Volume ID: 0x0
Volume Label (Boot Sector):
Volume Label (Root Directory):
File System Type Label: *3♦♦M|

Sectors before file system: 0

File System Layout (in sectors)
Total Range: 0 - 2879
* Reserved: 0 - 0
** Boot Sector: 0
* FAT 0: 1 - 9
* FAT 1: 10 - 18
* Data Area: 19 - 2879
** Root Directory: 19 - 32
** Cluster Area: 33 - 2879

METADATA INFORMATION
-----
Range: 2 - 45782
Root Directory: 2

CONTENT INFORMATION
-----
Sector Size: 512
Cluster Size: 512
Total Cluster Range: 2 - 2848

FAT CONTENTS (in sectors)
-----
33-236 (204) -> EOF
285-311 (27) -> EOF
```

Fig 1.14 Using the ffstat command to get more info about the disk

We then move to the **fls** command which lists files and directories, including deleted files, and shows their inode/MFT addresses.


```

(kali@kali)-[~/carvinglab]
$ fls -r Ch01InChap01.dd
r/r 5: Client Info.mdb
r/r * 8: Billing Letter.doc
r/r * 11: confirmation.txt
r/r 13: Income.xls
r/r * 15: letter1.txt
r/r * 17: Regrets.doc
v/v 45779: $MBR
v/v 45780: $FAT1
v/v 45781: $FAT2
V/V 45782: $OrphanFiles

(kali@kali)-[~/carvinglab]
$ 

```

Fig 1.15 using the fls command to get inodes and deleted files

Then we move to using the **icat** which Extracts/reconstructs a file from the image using its inode/file address.

```

(kali@kali)-[~/carvinglab]
$ fls -r -d Ch01InChap01.dd
r/r * 8: Billing Letter.doc
r/r * 11: confirmation.txt
r/r * 15: letter1.txt
r/r * 17: Regrets.doc

(kali@kali)-[~/carvinglab]
$ icat Ch01InChap01.dd 13 > recovered_file.jpg

(kali@kali)-[~/carvinglab]
$ ls
Ch01InChap01.dd  J_ub_law_hexdump.txt  J_ub_law.jpg  J_ub_law_reverse_dum.jpg  recovered_file.jpg

(kali@kali)-[~/carvinglab]
$ 

```

Fig 1.16 Extract and reconstructs an image using the inode number

Next we go through the raw contents of a specific **data block** from the image using blkcat

```

(kali@kali)-[~/carvinglab]
$ ls
Ch01InChap01.dd  J_ub_law_hexdump.txt  J_ub_law.jpg  J_ub_law_reverse_dum.jpg  recovered_file.jpg

(kali@kali)-[~/carvinglab]
$ blkcat Ch01InChap01.dd 33
Standard Jet DB*nb`      *U*+gr?+--+*****1*y**0**ç**âF***N***7]D***^(***`T*{6*~})*C*+34ay[**|**|**0J*1>`6_**b$*g*'D***e***F*x
**--

```

Fig 1.17 Using blkcat

Part D - Embedded File Discovery with Binwalk

For the Binwalk exercise, the **File_carving.docx** file wasn't made available and hence the manual and automatic extraction demonstrated using the file could not be carried out. I therefore proceeded with the Binwalk practice using the other supplied lab evidence.

Part E - Prepare the USB Image

First, we extract the USB Image **120M.7z**

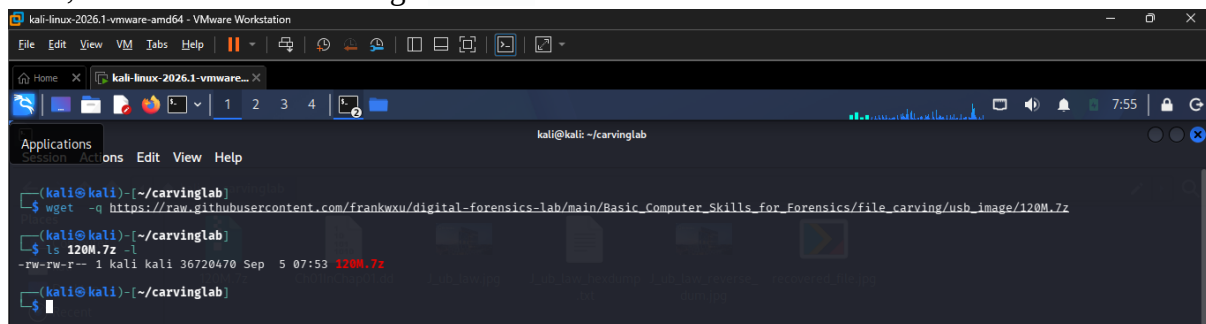


Fig 1.18 Extracted a USB Image and confirmed if it was downloaded

We then calculate hashes using the **hashdeep** command

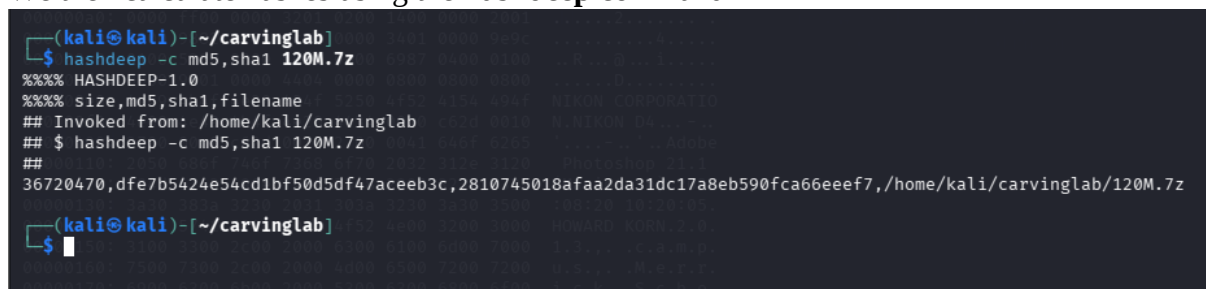


Fig 1.19 Calculated hashes (md5, sha1) using hashdeep

The content of the Zipped file is gotten by using the command 7z

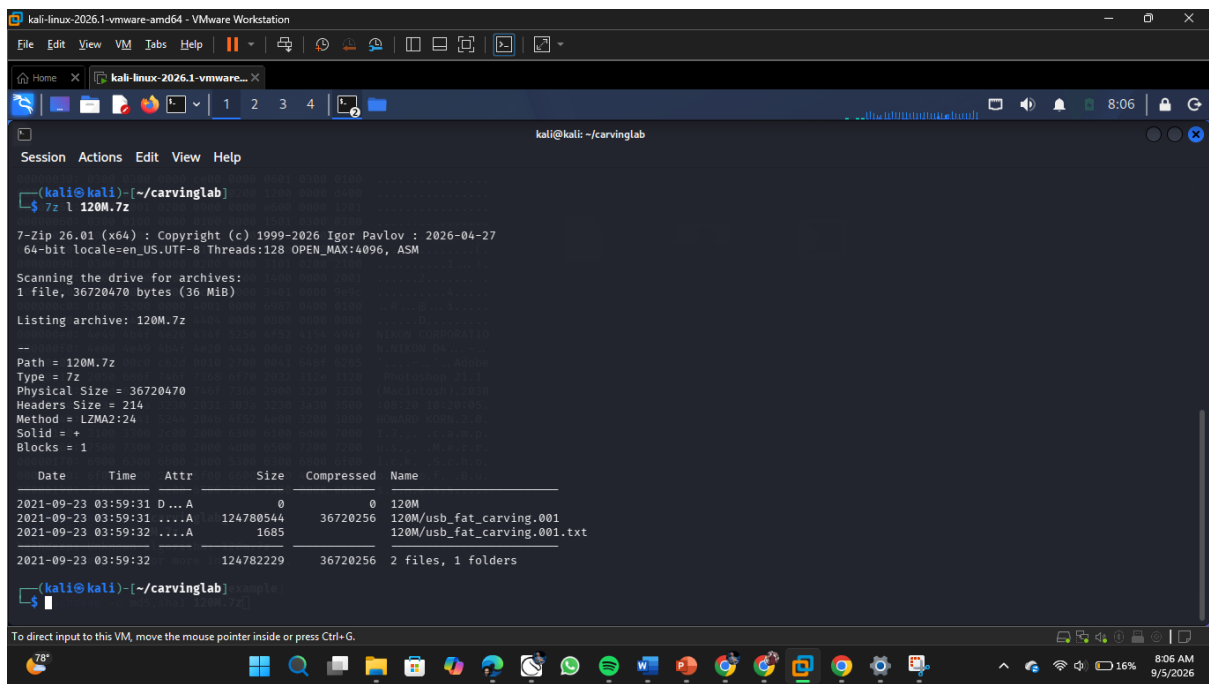


Fig 1.20 Gets the details of a zipped file

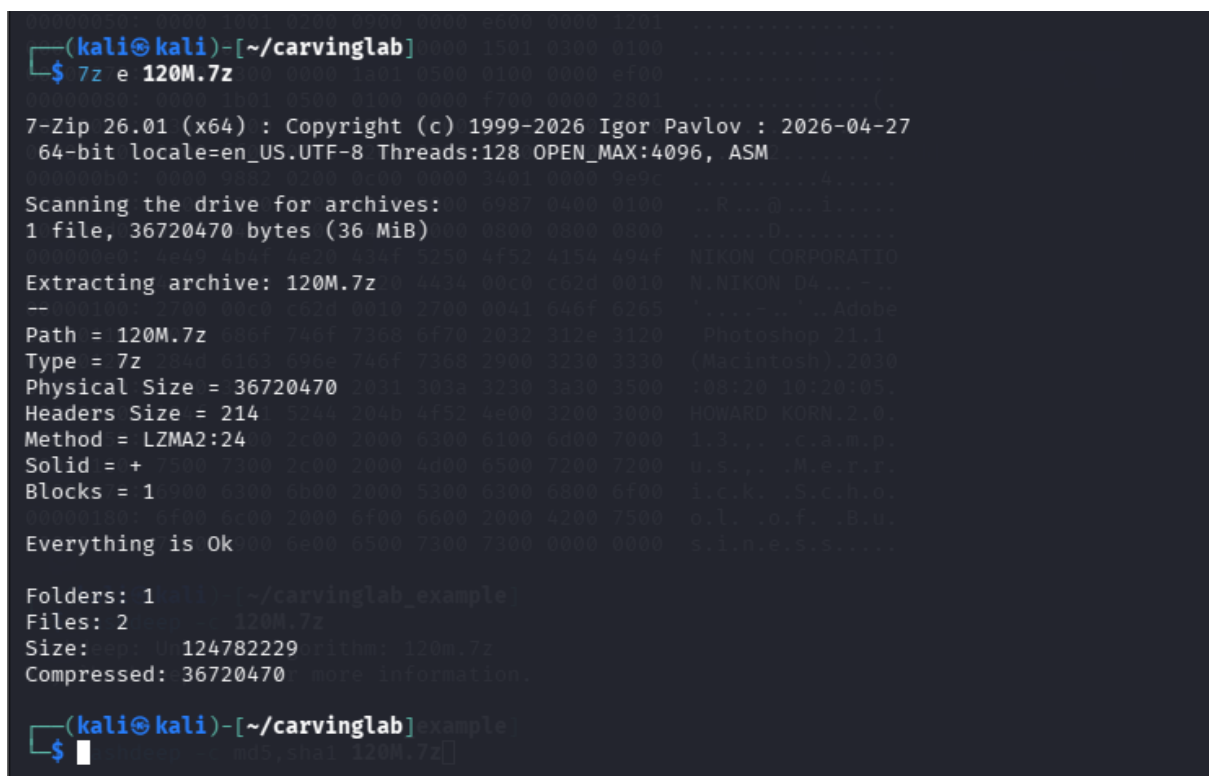


Fig 1.21 Lists the content of a zipped file and extracts files

We then verify the hashes of the file content **usb_file_carving.001.txt** using hashdeep and getting the checksum values.

```
(kali@kali)-[~/carvinglab]
$ hashdeep -c md5,sha1 usb_fat_carving.001
%%% HASHDEEP-1.0
%%% size,md5,sha1,filename
## Invoked from: /home/kali/carvinglab
## $ hashdeep -c md5,sha1 usb_fat_carving.001
##
124780544,ba4a1d0ba49f4a6667b00a3b3e85e604,bcc2d49fd49c9521ecb1739f6542c6bf327375ef,/home/kali/carvinglab/usb_fat_carving.001

(kali@kali)-[~/carvinglab]
$ cat usb_fat_carving.001.txt | grep "checksum"
MD5 checksum: ba4a1d0ba49f4a6667b00a3b3e85e604
SHA1 checksum: bcc2d49fd49c9521ecb1739f6542c6bf327375ef
MD5 checksum: ba4a1d0ba49f4a6667b00a3b3e85e604 : verified
SHA1 checksum: bcc2d49fd49c9521ecb1739f6542c6bf327375ef : verified

(kali@kali)-[~/carvinglab]
$
```

Fig 1.22 Verifying the Hashes of the file

Now we explore it using TSK tools and the fdisk was used to view the disk partitions and we see that the start offset to be 128 and the sectors and the size of the disk are also clearly stated

```
(kali@kali)-[~/carvinglab]
$ fdisk -l usb_fat_carving.001
Disk usb_fat_carving.001: 119 MiB, 124780544 bytes, 243712 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0xa1159a00

Device Boot Start End Sectors Size Id Type
usb_fat_carving.001p1 * 128 243711 243584 118.9M e W95 FAT16 (LBA)
```

Fig 1.22 Using fdisk to view disk partition

Using mmls we see that the start for the particular usb drive is 128

```
(kali@kali)-[~/carvinglab]
$ mmls usb_fat_carving.001
DOS Partition Table
Offset Sector: 0
Units are in 512-byte sectors

Slot Start End Length Description
000: Meta 0000000000 0000000000 0000000001 Primary Table (#0)
001: 0000000000 0000000127 0000000128 Unallocated
002: 000:000 0000000128 0000243711 0000243584 Win95 FAT16 (0x0e)
```

Fig 1.23 Using mmls to view the disk partition and get the offset

Part F - Carve Deleted Files with Scalpel

```
kali@kali: ~/carvinglab
Session Actions Edit View Help
(kali@kali)~$ fls -o 128 usb_fat_carving.001
r/r 3: USB (Volume Label Entry)
d/d 6: System Volume Information
r/r * 7: .est
r/r 10: .dropbox_device
d/d * 13: old_File_Carving_files
r/r * 15: B_ub_poe4.bmp
r/r * 18: B_zoom-eubie-mono.bmp
r/r * 21: BallardLab8.java
r/r * 24: brittLab10.java
r/r * 27: DO_example.doc
r/r * 30: DO_example2.doc
r/r * 33: G_BuiltForThis.gif
r/r * 35: G_zoom-sc.gif
r/r * 37: H_Form.html
r/r * 39: H_hello.html
r/r * 41: J_ub_law.jpg
r/r * 44: J_ub_night.jpg
r/r * 47: nps-2008-jean_outlook.pst
r/r * 50: P_CAS-zoom-6.png
r/r * 53: P_MSB_1_zoom.png
r/r * 57: pd_Evidence_search_techniques.pdf
r/r * 61: pd_Forensic_Report_Template.pdf
r/r * 64: pp_Number_Systems.pptx
r/r * 67: pp_one_page.pptx
r/r 69: readme.docx
r/r 70: readme.txt
r/r * 71: _tff.1.rtf
r/r * 74: T_Eubie-iphone-5-8.tiff
r/r * 78: T_Youknows-iphone-5-8.tiff
r/r * 81: W_axloadcomplete.wav
r/r * 84: W_speaker_test_sound.wav
r/r * 86: Z_file.zip
r/r * 88: 江莎莎行踪轨迹.doc
r/r * 91: nps-2008-jean_outlook.pst
v/v 3889667: $MBR
v/v 3889668: $FAT1
v/v 3889669: $FAT2
v/v 3889670: $OrphanFiles
```

Fig 1.24 Taking note of the deleted files using fls

Then now we need to configure the scalpel file to be able to carve

```
kali@kali: ~/carvinglab
CPU usage: 2.0%
# Here is an example of how to use the no extension option. Any files
# beginning with the string "FOREMOST" are carved and no file extensions
# are used. No footer is defined and the max carve size is 1000 bytes.
#
# NONE y 1000 FOREMOST
#
# GRAPHICS FILES
#
# AOL ART files
# art y 150000 \x4a\x47\x04\x0e \xc7\xc7\xcb
# art y 150000 \x4a\x47\x03\x0e \xd0\xcb\x00\x00
#
# GIF and JPG files (very common)
# gif y 5000000 \x47\x49\x46\x38\x37\x61 \x00\x3b
# gif y 5000000 \x47\x49\x46\x38\x39\x61 \x00\x3b
# jpg y 5242880 \xff\xd8\xff???Exif \xff\xd9 REVERSE
# jpg y 5242880 \xff\xd8\xff???JFIF \xff\xd9 REVERSE
#
# PNG
# png y 20000000 \x50\x4e\x47? \xff\xfc\xfd\xfe
#
# BMP (used by MSWindows, use only if you have reason to think there are
# BMP files worth digging for. This often kicks back a lot of false
# positives - carvinglab example
# /etc/scalpel/scalpel.conf
# bmp y 100000 BM??\x00\x00\x00
#
# TIFF
# tif y 20000000 \x49\x49\x2a\x00
# tif y 20000000 \x4d\x4d\x00\x2a
#
# ANIMATION FILES
"/etc/scalpel/scalpel.conf" [readonly] 240L, 8667B 87,1-8 33%
```

Fig 1.25 Inside the Scalpel config file and uncommented the jpg file for carving

We then create a new output file to save the content we want to carve into

```
kali@kali: ~/carvinglab
$ vim /etc/scalpel/scalpel.conf
(kali@kali)~/carvinglab
$ scalpel usb_fat_carving.001 -o output_new
Scalpel version 1.60
Written by Golden G. Richard III, based on Foremost 0.69.
Opening target "/home/kali/carvinglab/usb_fat_carving.001"
Image file pass 1/2.
usb_fat_carving.001: 100.0% |*****| 119.0 MB 00:00 ETA
Allocating work queues...
Work queues allocation complete. Building carve lists...
Carve lists built. Workload:
jpg with header "\xff\xd8\xff\x3f\x3f\x45\x78\x69\x66" and footer "\xff\xd9" -> 8 files
jpg with header "\xff\xd8\xff\x3f\x3f\x4a\x46\x49\x46" and footer "\xff\xd9" -> 9 files
Carving files from image.
Image file pass 2/2.
usb_fat_carving.001: 100.0% |*****| 119.0 MB 00:00 ETA
Processing of image file complete. Cleaning up...
Done.
Scalpel is done, files carved = 17, elapsed = 1 seconds.
(kali@kali)~/carvinglab
$
```

Fig 1.26 finished carving the file into the new_output file

```
(kali@kali)~/carvinglab
$ tree output_new
output_new
├── audit.txt
├── jpg-0-0
│   ├── 00000000.jpg
│   ├── 00000001.jpg
│   ├── 00000002.jpg
│   ├── 00000003.jpg
│   ├── 00000004.jpg
│   ├── 00000005.jpg
│   ├── 00000006.jpg
│   ├── 00000007.jpg
├── jpg-1-0
│   ├── 00000008.jpg
│   ├── 00000009.jpg
│   ├── 00000010.jpg
│   ├── 00000011.jpg
│   ├── 00000012.jpg
│   ├── 00000013.jpg
│   ├── 00000014.jpg
│   ├── 00000015.jpg
│   └── 00000016.jpg
3 directories, 18 files
(kali@kali)~/carvinglab
$
```

Fig 1.27 Using tree to check the files in the carved file

Now we review the audit.txt file to see the list of files that were carved in the carving operation using scalpel.


```
(kali@kali)-[~/carvinglab]
$ cat output_new/audit.txt

Scalpel version 1.60 audit file
Started at Sat Sep  5 08:38:02 2026
Command line:
scalpel usb_fat_carving.001 -o output_new

Output directory: /home/kali/carvinglab/output_new
Configuration file: /etc/scalpel/scalpel.conf

Opening target "/home/kali/carvinglab/usb_fat_carving.001"

The following files were carved:
File          Start      Chop      Length      Extracted From
00000010.jpg  3796992    NO        3148500      usb_fat_carving.001
00000009.jpg  425822    NO        4875802      usb_fat_carving.001
00000008.jpg  335872    NO        4965752      usb_fat_carving.001
00000002.jpg  6938624    NO        6868         usb_fat_carving.001
00000001.jpg  5285888    NO        1659604      usb_fat_carving.001
00000000.jpg  3897344    NO        3048148      usb_fat_carving.001
00000004.jpg  15427584  NO        4223822      usb_fat_carving.001
00000003.jpg  12881920  NO        4256310      usb_fat_carving.001
00000006.jpg  19638272  NO        5229050      usb_fat_carving.001
00000005.jpg  17121280  NO        4248530      usb_fat_carving.001
00000007.jpg  21358592  NO        5183267      usb_fat_carving.001
00000016.jpg  36671488  NO        2889262      usb_fat_carving.001
00000015.jpg  36571136  NO        2989614      usb_fat_carving.001
00000014.jpg  36393610  NO        3167140      usb_fat_carving.001
00000013.jpg  36352448  NO        3208302      usb_fat_carving.001
00000012.jpg  35590996  NO        3969754      usb_fat_carving.001
00000011.jpg  35350242  NO        4210508      usb_fat_carving.001

Completed at Sat Sep  5 08:38:03 2026

(kali@kali)-[~/carvinglab]
```

Fig 1.28 Checking the Carved files from the Audit.txt

Now checking the files

```
File Edit View VM Tabs Help CPU usage: 2.0%
kali-linux-2026.1-vmware... 52
kali@kali: ~/carvinglab
Session Actions Edit View Help
20M.7z J_ub_low_hexdump.txt J_ub_low_reverse_dum.jpg recovered_file.jpg usb_fat_carving.001.txt
(kali@kali)-[~/carvinglab]
$ cat output_new/audit.txt

Scalpel version 1.60 audit file
Started at Sat Sep  5 08:38:02 2026
Command line:
scalpel usb_fat_carving.001 -o output_new

Output directory: /home/kali/carvinglab/output_new
Configuration file: /etc/scalpel/scalpel.conf

Opening target "/home/kali/carvinglab/usb_fat_carving.001"

The following files were carved:
File          Start      Chop      Length      Extracted From
00000010.jpg  3796992    NO        3148500      usb_fat_carving.001
00000009.jpg  425822    NO        4875802      usb_fat_carving.001
00000008.jpg  335872    NO        4965752      usb_fat_carving.001
00000002.jpg  6938624    NO        6868         usb_fat_carving.001
00000001.jpg  5285888    NO        1659604      usb_fat_carving.001
00000000.jpg  3897344    NO        3048148      usb_fat_carving.001
00000004.jpg  15427584  NO        4223822      usb_fat_carving.001
00000003.jpg  12881920  NO        4256310      usb_fat_carving.001
00000006.jpg  19638272  NO        5229050      usb_fat_carving.001
00000005.jpg  17121280  NO        4248530      usb_fat_carving.001
00000007.jpg  21358592  NO        5183267      usb_fat_carving.001
00000016.jpg  36671488  NO        2889262      usb_fat_carving.001
00000015.jpg  36571136  NO        2989614      usb_fat_carving.001
00000014.jpg  36393610  NO        3167140      usb_fat_carving.001
00000013.jpg  36352448  NO        3208302      usb_fat_carving.001
00000012.jpg  35590996  NO        3969754      usb_fat_carving.001
00000011.jpg  35350242  NO        4210508      usb_fat_carving.001

Completed at Sat Sep  5 08:38:03 2026

(kali@kali)-[~/carvinglab]
$ display -resize 500x600 output_new/jpg-1-0/00000008.jpg

ImageMagick: 00000008.jpg
```

Fig 1.29 First Carved JPG image from the audit file

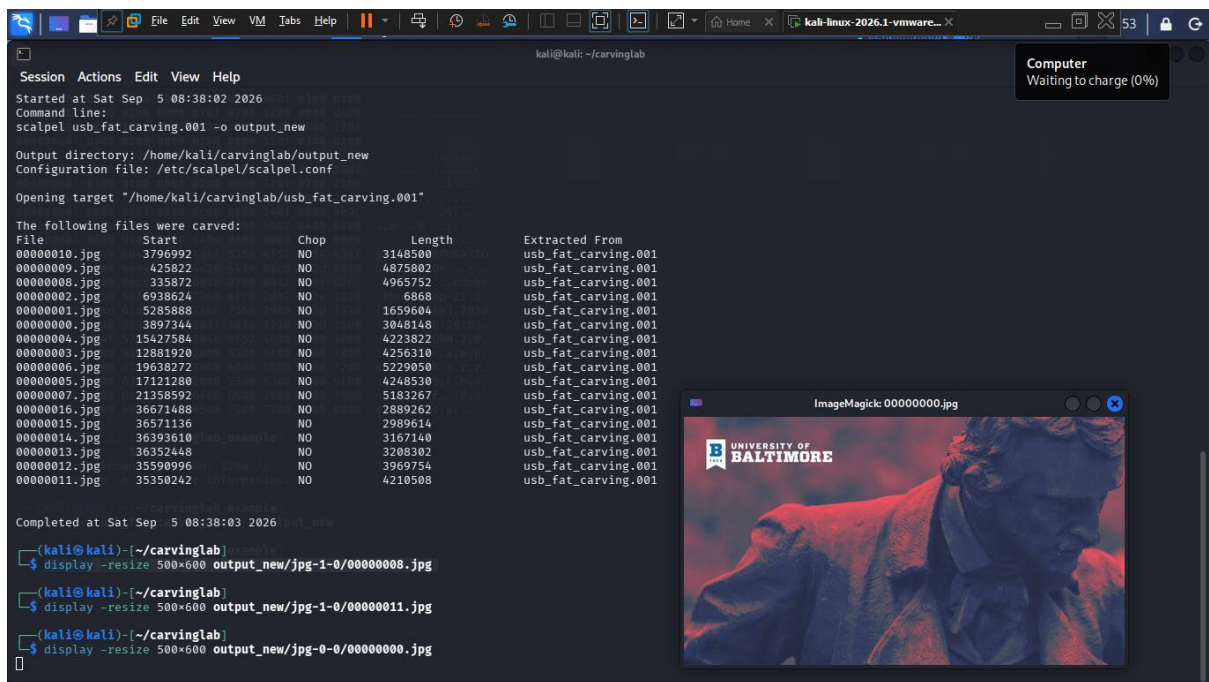


Fig 1.30 Second Carved JPG image from the audit file

In summary this lab focused on **digital forensic data carving and file recovery**. It involved examining binary data with **xxd**, identifying file signatures, reconstructing files from hexadecimal data, using **The Sleuth Kit (TSK)** to examine filesystem structures and recover files, and using tools such as **Binwalk** and **Scalpel** for file extraction and recovery. Finally, recovered files were verified using **MD5** and **SHA-256** hashes to ensure their integrity.